Project 1: RAG Chatbot (FAISS-Based)

1. Introduction

The Retrieval-Augmented Generation (RAG) Chatbot is designed to answer user queries based on uploaded documents by combining information retrieval with generative AI. Instead of relying solely on pre-trained knowledge, the system retrieves relevant document content and uses it to generate accurate and context-aware responses.

This implementation uses a local vector database (FAISS), making it efficient and suitable for offline or low-latency environments.

2. Objective

The objectives of this project are:

- To develop a document-based question answering system
- To improve response accuracy using retrieval-based techniques
- To implement semantic search using vector embeddings
- To provide a simple and interactive user interface
- To demonstrate a lightweight RAG pipeline using FAISS

3. Technologies and Libraries Used

Programming Language: Python

Frontend: Streamlit

Embedding Model: SentenceTransformers (all-MiniLM-L6-v2)

Vector Database: FAISS (Facebook AI Similarity Search)

LLM Integration: Gemini API

Supporting Libraries:

- NumPy
- tempfile
- pathlib
- dotenv (for environment variables)

4. Architecture

System Workflow

1. User uploads a text document

2. Document is processed and split into chunks
3. Text chunks are converted into embeddings
4. Embeddings are stored in a FAISS index
5. User enters a query
6. System retrieves top-K relevant chunks
7. LLM generates answer using retrieved context

Components

- **User Interface Layer:** Streamlit
- **Embedding Layer:** Converts text into vectors
- **Storage Layer:** FAISS index (local vector store)
- **Retrieval Layer:** Fetches relevant document chunks
- **Generation Layer:** Produces final response using LLM

5. Features

- Upload and process text documents
- Semantic similarity-based retrieval
- Fast local vector search using FAISS
- Adjustable Top-K retrieval results
- Context-aware response generation
- Session-based indexing for efficiency
- Reset functionality for new document uploads
- Cached responses to optimize performance
- Clean and structured user interface

6. Code Implementation

```
import streamlit as st
import tempfile
from pathlib import Path

from embeddings import load_model
from ingestion import load_documents_from_file
```

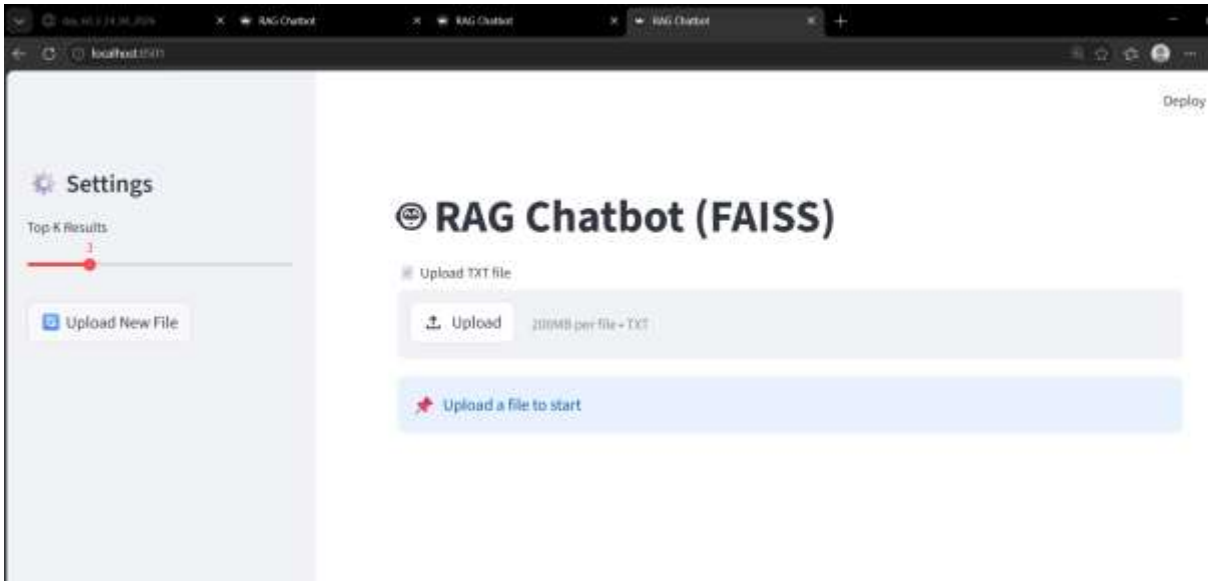
```
from vector_store import create_faiss_index
from retriever import retrieve
from llm import ask_llm

st.set_page_config(page_title="RAG Chatbot", layout="wide")

st.title("🤖 RAG Chatbot (FAISS)")

st.sidebar.title("⚙️ Settings")
top_k = st.sidebar.slider("Top K Results", 1, 10, 3)

uploaded_file = st.file_uploader("📄 Upload TXT file", type=["txt"])

# SESSION STATE INIT
if "index" not in st.session_state:
    st.session_state.index = None
    st.session_state.documents = None
    st.session_state.model = None

#  RESET BUTTON
if st.sidebar.button(" Upload New File"):
    st.session_state.index = None
    st.session_state.documents = None
    st.session_state.model = None

#  CACHE LLM RESPONSE
@st.cache_data(show_spinner=False)
def cached_llm(query, context_tuple):
    return ask_llm(query, [{"document": doc} for doc in context_tuple])

if uploaded_file:
```

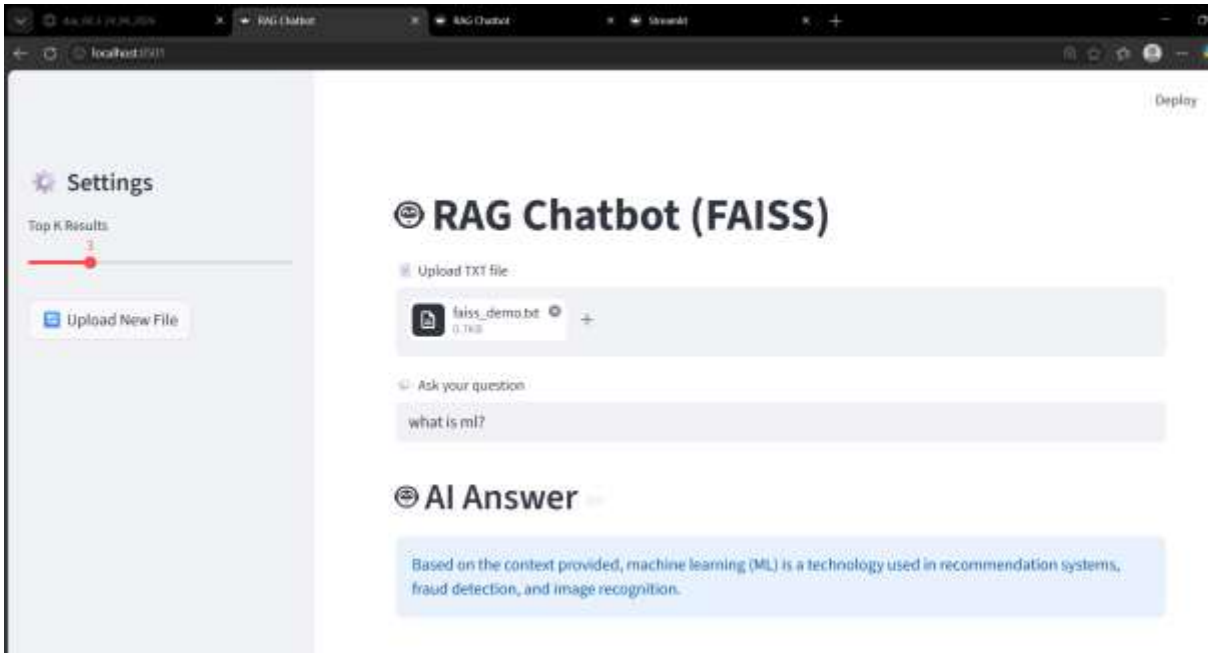
```
if st.session_state.index is None:

    temp_dir = tempfile.gettempdir()
    filepath = Path(temp_dir) / uploaded_file.name

    with open(filepath, "wb") as f:
        f.write(uploaded_file.getbuffer())

    with st.spinner("Loading embedding model..."):
        st.session_state.model = load_model()

    documents = load_documents_from_file(filepath)
    st.session_state.documents = documents

    st.success(f"Loaded {len(documents)} chunks")

    with st.spinner("Creating FAISS index..."):
        st.session_state.index = create_faiss_index(
            documents,
            st.session_state.model
        )

query = st.text_input("💬 Ask your question")

if query and st.session_state.index is not None:

    with st.spinner("Retrieving relevant documents..."):
        results = retrieve(
            query,
            st.session_state.model,
```

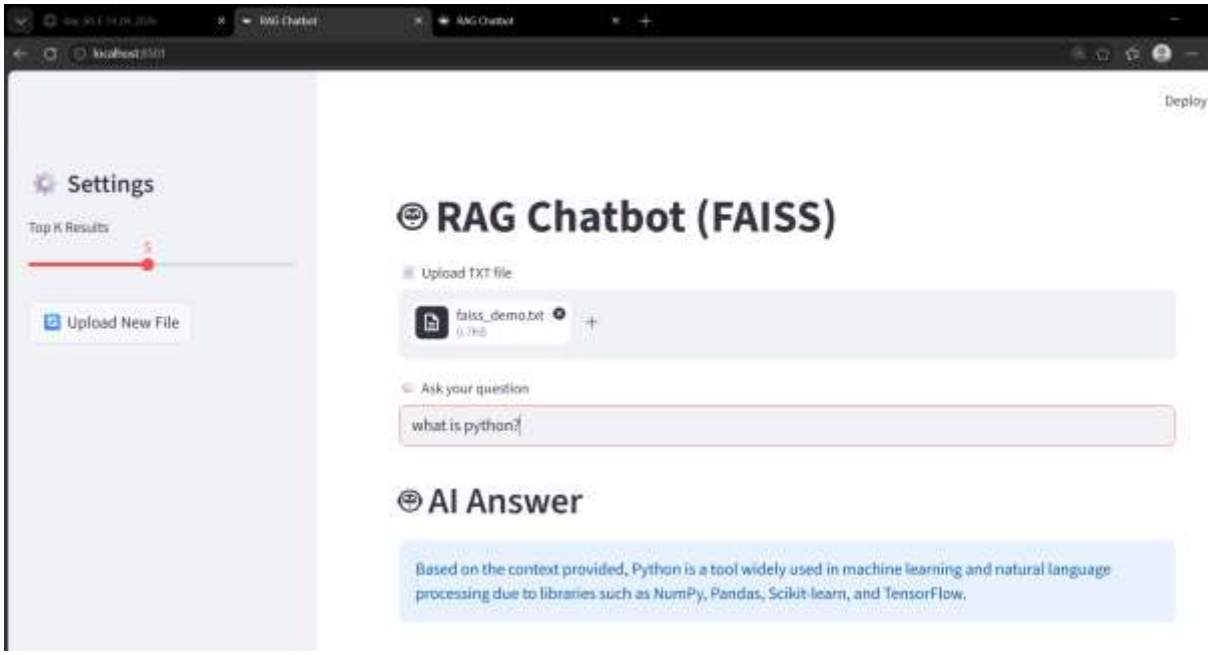
```
        st.session_state.index,
        st.session_state.documents,
        top_k
    )

context_tuple = tuple([r["document"] for r in results])

with st.spinner("Generating answer..."):
    answer = cached_llm(query, context_tuple)

# AI Answer (cleaner look)
st.markdown("## 🤖 AI Answer")
st.info(answer)

# Retrieved Context (FIXED UI)
st.markdown("## 🔎 Retrieved Context")

for i, r in enumerate(results):
    if i == 0:
        title = "Best Match"
    else:
        title = f"🔹 Result {i+1}"

    st.markdown(f"### {title}")

    with st.container():
        st.markdown(f"""
        <div style="
            background-color:#f1f5f9;
            padding:15px;
            border-radius:10px;
```

```
            margin-bottom:10px;
            border:1px solid #cbd5e1;
            color:#000000;
        ">
            <b>Score:</b> {r['score']:.4f}<br><br>
            {r['document']}
        </div>
        """, unsafe_allow_html=True)

else:
    st.info("📌 Upload a file to start")
```

7. Output







8. Conclusion

The FAISS-based RAG Chatbot demonstrates how local vector search combined with generative AI can produce accurate and context-aware responses without relying on external vector databases. This makes the system efficient, scalable, and suitable for applications where low latency and data privacy are important.

The project can be further enhanced with multi-document support, metadata filtering, and conversational memory for more advanced use cases.

Project 2: RAG Chatbot (Pinecone)

1. Introduction

The Retrieval-Augmented Generation (RAG) Chatbot is an intelligent system designed to answer user queries based on uploaded documents. Instead of relying only on a language model's pre-trained knowledge, the system retrieves relevant information from a document and uses it to generate accurate, context-aware responses.

This approach combines **semantic search** with **generative AI**, making the chatbot more reliable and grounded in real data.

2. Objective

The main objectives of this project are:

- To build a document-based intelligent chatbot
- To improve answer accuracy using retrieval mechanisms
- To implement semantic search using embeddings
- To create an interactive web interface
- To demonstrate real-world application of RAG architecture

3. Technologies & Libraries Used (Tech Stack)

Programming Language: Python

Frontend: Streamlit (for interactive UI)

Backend & Processing: Python-based modular architecture

Embedding Model: SentenceTransformers (all-MiniLM-L6-v2)

Vector Database: Pinecone (for storing and retrieving embeddings)

LLM Integration: Gemini API (for generating responses)

Supporting Libraries:

- NumPy (vector operations)
- Pandas (data handling, if used)
- dotenv (environment variable management)
- tempfile & pathlib (file handling)

4. Architecture

System Workflow:

1. User uploads a text document
2. Document is split into smaller chunks
3. Chunks are converted into vector embeddings
4. Embeddings are stored in Pinecone
5. User enters a query
6. System retrieves Top-K relevant chunks
7. LLM generates answer using retrieved context

Components:

- **UI Layer**: Streamlit interface
- **Embedding Layer**: Converts text to vectors
- **Storage Layer**: Pinecone vector database
- **Retrieval Layer**: Fetches relevant data
- **Generation Layer**: Produces final response

5. Features

- Document upload support (.txt)
- Semantic similarity-based retrieval
- Fast vector search using Pinecone
- Adjustable Top-K results
- Context-aware answer generation
- One-time indexing optimization
- Automatic clearing of previous data
- Interactive chatbot interface

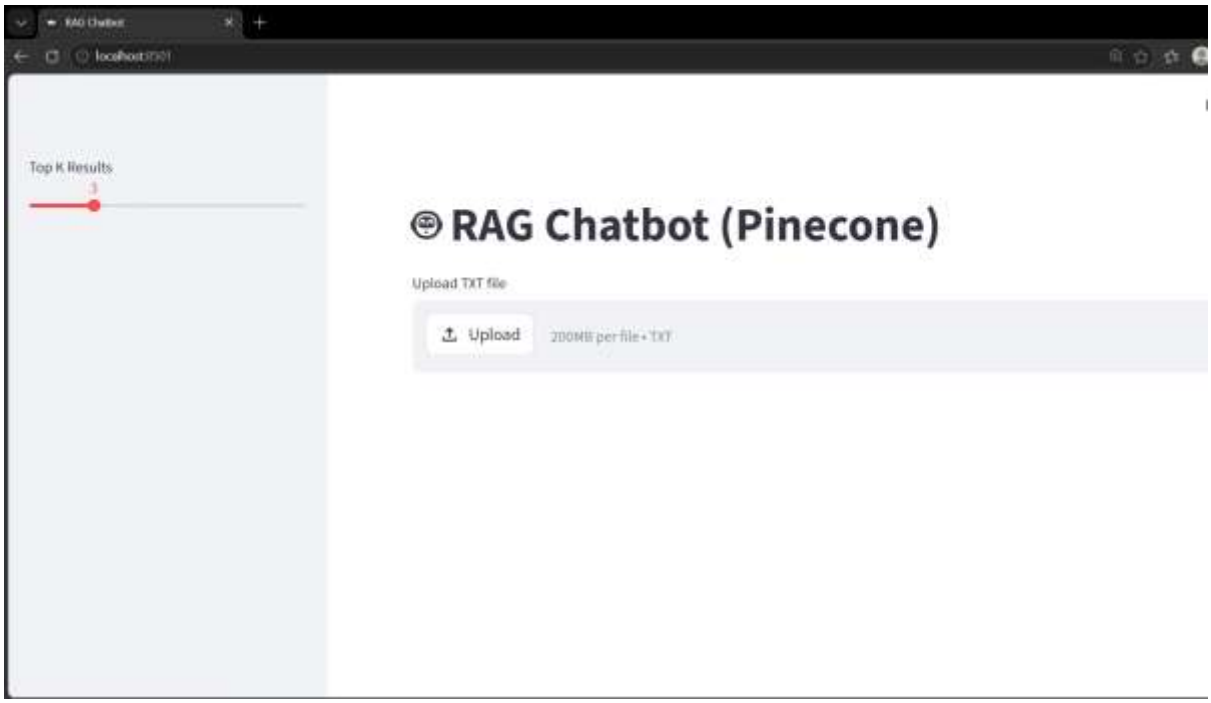
6. Code Implementation

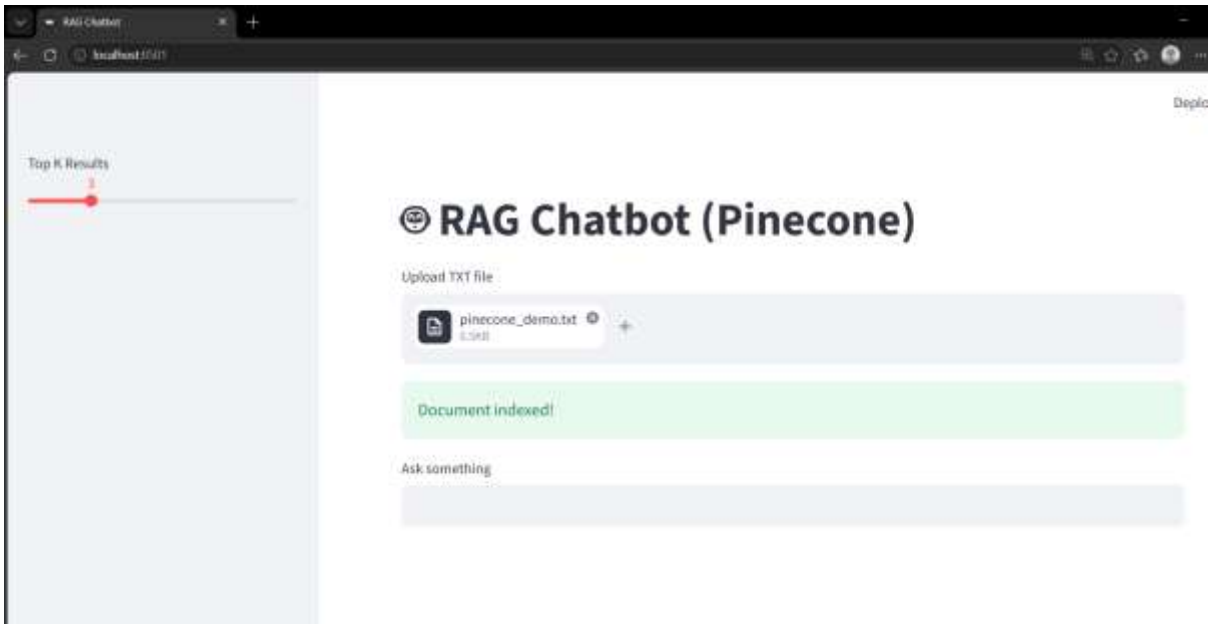
```
import streamlit as st

import tempfile

from pathlib import Path
```

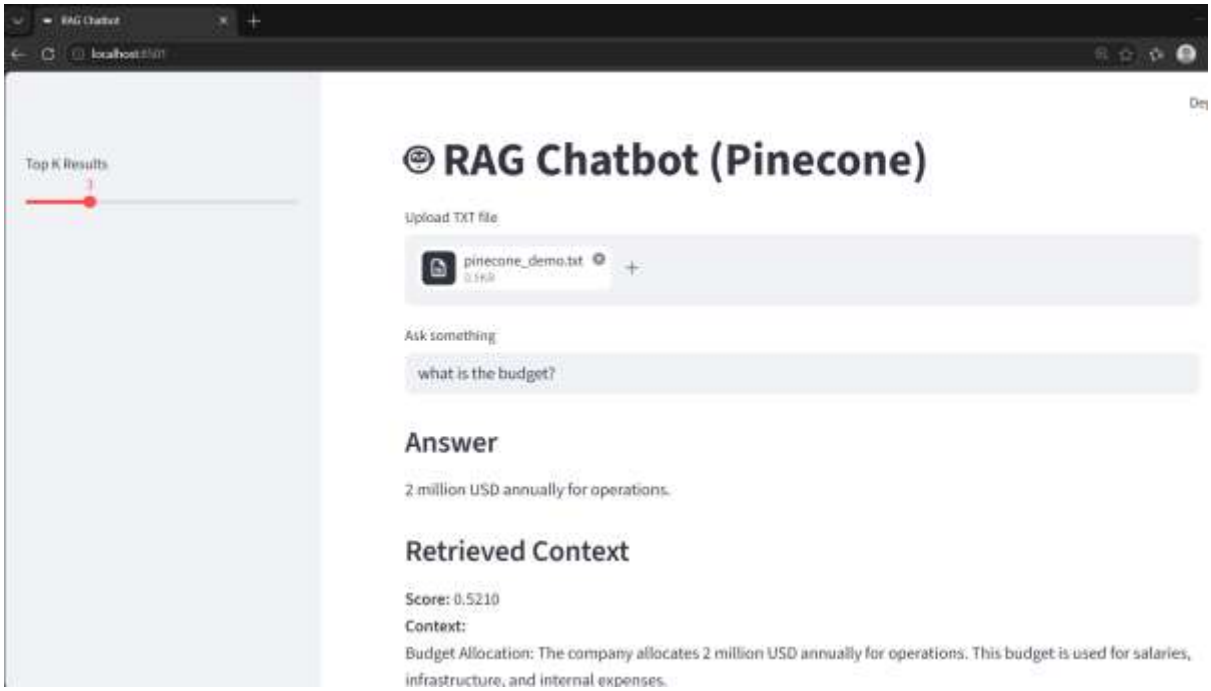
```
from embeddings import load_model
from ingestion import load_documents_from_file, upload_to_pinecone
from vector_store import init_pinecone
from retriever import retrieve
from llm import ask_llm

st.set_page_config(page_title="RAG Chatbot", layout="wide")

st.title("🤖 RAG Chatbot (Pinecone)")

top_k = st.sidebar.slider("Top K Results", 1, 10, 3)

# Session state flags
if "indexed" not in st.session_state:
    st.session_state.indexed = False

uploaded_file = st.file_uploader("Upload TXT file", type=["txt"])

if uploaded_file:

    temp_path = Path(tempfile.gettempdir()) / uploaded_file.name

    with open(temp_path, "wb") as f:
        f.write(uploaded_file.getbuffer())

    model = load_model()
    index = init_pinecone()

    #  ONLY RUN ONCE
    if not st.session_state.indexed:
        docs = load_documents_from_file(temp_path)
```

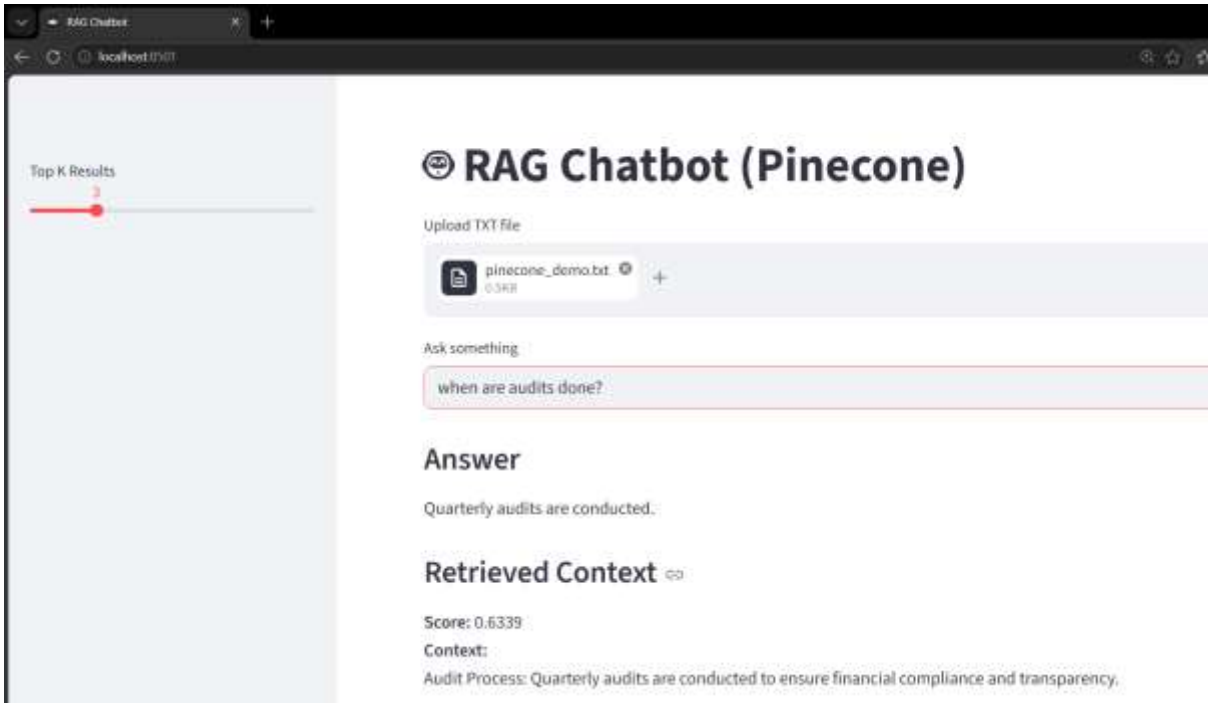
```
# CLEAR OLD DATA (VERY IMPORTANT)
    index.delete(delete_all=True)

    upload_to_pinecone(docs, model, index)

    st.session_state.indexed = True
    st.success("Document indexed!")

query = st.text_input("Ask something")

if query:
   retrieved_docs = retrieve(query, model, index, top_k)

   context_texts = [doc["text"] for doc in retrieved_docs]

   answer = ask_llm(query, context_texts)

   st.subheader("Answer")
   st.write(answer)

   st.subheader("Retrieved Context")

   for d in retrieved_docs:
        st.markdown(f"""
   **Score:** {d['score']:.4f}
   **Context:**
   {d['text']}
   """)
```

7.Output


Top K Results
3
RAG Chatbot (Pinecone)
Upload TXT file
Upload


Top K Results
3
RAG Chatbot (Pinecone)
Upload TXT file
pinecone_demo.txt
Document indexed!
Ask something





8. Conclusion

The RAG Chatbot demonstrates how integrating **vector databases with large language models** significantly improves the quality of responses. By grounding answers in retrieved context, the system reduces hallucinations and delivers more accurate results.

This project can be extended into enterprise applications such as **knowledge assistants, customer support bots, and educational tools**.